

Reviewer Pack — Minimal Rank of Quadratic Residues Bounds XOR-AND Tree Width

Entry 67af54f5d24b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 18:37:54 UTC

Minimal Rank of Quadratic Residues Bounds XOR-AND Tree Width

Entry ID: 67af54f5d24b **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-27 18:37:54 UTC

1. Conjecture

Field A (mathematical branch): Number Theory (Quadratic Residues) **Field B** (complexity object): Complexity Theory: XOR-AND Tree Width

Statement:

[‘For a given Boolean function f , the XOR-AND tree width $t(f)$ is upper-bounded by the number of quadratic residues modulo p for some prime p , i.e., $E[t(f)] \leq \sqrt{p}$.’, ‘If there exists a Boolean function f such that $t(f) > \sqrt{p}$ for all primes p , then f is not computable in polynomial time.’, ‘The XOR-AND tree width of the majority function on n bits is at least $\sqrt{n}$.’]

Rationale (proposer’s reasoning):

[‘Quadratic residues have been used to study number-theoretic properties and their connection to Boolean functions could reveal new insights into circuit complexity.’, ‘Previous work has shown that certain number-theoretic invariants can be linked to communication complexity, suggesting a potential bridge to XOR-AND tree width.’, ‘This conjecture aims to establish a direct quantitative relationship between these two fields, which if true, would provide new tools for understanding the limits of computation.’]

Taxonomy category: ACC_SIPSER (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 99eacd960dbf04cd

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

To support the conjecture, the average XOR-AND tree width $t(f)$ must be less than or equal to $\sqrt{p}$ for at least 80% of the 30 random seeds, where p is a prime greater than $t(f)$. To falsify, more than 20% of the seeds must have $t(f) > \sqrt{p}$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.3s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_boolean_function(n, m):
        return [[random.randint(0, 1) for _ in range(m)] for _ in range(2**n)]

    def xor_and_tree_width(f):
        n = len(f)
        m = len(f[0])

        def evaluate(x, y):
            if x == y:
                return f[x][y]
            else:
                return 1

        def xor_and(a, b):
            result = 0
            for i in range(m):
                result |= evaluate(a, i) & evaluate(b, i)
            return result

        def count_ones(x):
            return bin(x).count('1')

        def dfs(node, visited):
            if node in visited:
                return 0
            visited.add(node)
            max_width = 0
```

```

        for child in range(m):
            if evaluate(node, child) == 1:
                max_width = max(max_width, dfs(child, visited))
        return 1 + max_width

visited = set()
return dfs(0, visited)

def count_quadratic_residues(p, values):
    residues = [i**2 % p for i in range(p)]
    return sum(value in residues for value in values)

n = 40
m = n
f = generate_random_boolean_function(n, m)
t_f = xor_and_tree_width(f)

if t_f == 0:
    return {
        "metric_name": "XOR-AND tree width",
        "metric_value": t_f,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

p = next((i for i in range(t_f + 1, 2 * t_f) if all(i % j != 0 for j in range(2, int(math.sqrt(i)) + 1))), None)
if p is None:
    return {
        "metric_name": "XOR-AND tree width",
        "metric_value": t_f,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

values = [f[i][j] for i in range(2*n) for j in range(m)]
num_residues = count_quadratic_residues(p, values)

return {
    "metric_name": "XOR-AND tree width",
    "metric_value": t_f,
    "instances_tested": 1,
    "conjecture_holds": t_f <= math.sqrt(p),
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

```

```

if all("conjecture_holds" in r and r["conjecture_holds"] for r in results):
    mean_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = 1.0
    print(f"RESULT: SUPPORTED mean={mean_value} std=0 support_fraction={support_fraction}")
elif any("conjecture_holds" in r and not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if "conjecture_holds" not in result)
    print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means we cannot verify the conjecture's support or falsification. | next: Run the test again with increased time limits to ensure it completes and produces results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12244
2	preregistration	ollama_remote	glm4:latest	0	0	5621
3	novelty	ollama_remote	glm4:latest	0	0	4834
4	novelty	ollama_remote	glm4:latest	0	0	5080
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12732
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12555
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7799
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10467
9	judge	ollama_remote	glm4:latest	0	0	14783

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 86114 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in [research/audit/67af54f5d24b.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/67af54f5d24b.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/67af54f5d24b.tar.gz` (if generated)